

1) Import Libraries

```
In [1]: import pandas as pd
import numpy as np
import logging
import shap
import matplotlib.pyplot as plt
import joblib
import warnings
from scipy import stats
from sklearn.model_selection import TimeSeriesSplit, train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_percentage_error, mean_squared_error, r2_score
from sklearn.feature_selection import SelectKBest, f_regression
```

2) Load and Clean Data

```
In [2]: # Read the data
df = pd.read_csv('Data/stockdata.csv')
print(f"Initial data shape: {df.shape}")

# Store original data for future predictions
current_data = df.copy()

# Encode categorical variables
categorical_columns = ['PE Expanding', 'Book Value Expanding', 'Reducing Debt',
                      'Increasing Margin', 'Valuation', 'Bu Model',
                      'Growth', 'Financial Health ', 'Ownership']

# Replace 'N' with 0 for binary columns
binary_columns = ['PE Expanding', 'Book Value Expanding', 'Reducing Debt', 'Increasing Margin']
for col in binary_columns:
    df[col] = df[col].map({'Y': 1, 'N': 0})
    current_data[col] = current_data[col].map({'Y': 1, 'N': 0})

# Replace 'N' with 0 for metric columns
metric_columns = ['Valuation', 'B Model', 'Growth', 'Financial Health ', 'Ownership']
for col in metric_columns:
    df[col] = pd.to_numeric(df[col].replace('N', '0'))
    current_data[col] = pd.to_numeric(current_data[col].replace('N', '0'))

# Store company names for later use
company_names = current_data['Name'].copy()
```

Initial data shape: (2972, 48)

3) Feature Engineering

```
In [3]: # Identify numeric columns (excluding 'Name' and target variable)
numeric_columns = df.select_dtypes(include=[np.number]).columns.tolist()
numeric_columns.remove('Return over 1year') # Remove target variable
print(f"Number of features: {len(numeric_columns)}")

# Handle missing values
df[numeric_columns] = df[numeric_columns].fillna(df[numeric_columns].median())
current_data[numeric_columns] = current_data[numeric_columns].fillna(df[numeric_columns].median())
```

```
# Fit scaler on all historical data
scaler = StandardScaler()
scaler.fit(df[numeric_columns])

# Transform both historical and current data
df[numeric_columns] = scaler.transform(df[numeric_columns])
current_data[numeric_columns] = scaler.transform(current_data[numeric_columns])
```

Number of features: 46

4)Prepare Target Variable

```
In [4]: # Convert target variable
target_variable = 'Return over 1year'

# Remove extreme returns
df = df[df[target_variable] > -100]

# Convert to decimal
df['Return_decimal'] = df[target_variable] / 100

# Apply Log transformation
df['Target'] = np.log1p(df['Return_decimal'])

# Remove any infinities or NaNs
df = df.replace([np.inf, -np.inf], np.nan).dropna(subset=['Target'])
print(f"Final shape after target preparation: {df.shape}")

# Cell 5: Feature Selection
X = df[numeric_columns]
y = df['Target']

# Select all features
selector = SelectKBest(score_func=f_regression, k='all')
selector.fit(X, y) # Fit on all historical data
selected_features = X.columns[selector.get_support()].tolist()

# Transform both historical and current data
X = pd.DataFrame(selector.transform(X), columns=selected_features, index=X.index)
X_current = pd.DataFrame(selector.transform(current_data[numeric_columns]),
                        columns=selected_features,
                        index=current_data.index)

print(f"Selected features shape: {X.shape}")
```

Final shape after target preparation: (2688, 50)

Selected features shape: (2688, 46)

5)Split Data

```
In [5]: # Use TimeSeriesSplit to maintain temporal order
tscv = TimeSeriesSplit(n_splits=3)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, shuffle=False)
print(f"Training set shape: {X_train.shape}")
print(f"Testing set shape: {X_test.shape}")
```

Training set shape: (2150, 46)

Testing set shape: (538, 46)

6)Train Random Forest Model with GridSearchCV

```

In [6]: # Define parameter grid (adjusted for GridSearchCV)
param_grid = {
    'n_estimators': [100, 200, 300, 400, 500],
    'max_depth': [None, 5, 7, 9, 11, 13],
    'min_samples_split': [2, 4, 6, 8],
    'min_samples_leaf': [1, 2, 4],
    'max_features': ['sqrt', 'log2']
}

# Create base model
rf = RandomForestRegressor(random_state=42)

# Create GridSearchCV object with updated param_grid
grid_search = GridSearchCV(
    estimator=rf,
    param_grid=param_grid,
    cv=tscv,
    n_jobs=-1,
    verbose=2,
    scoring='neg_mean_squared_error'
)

# Fit on all historical data
print("Starting GridSearchCV with updated parameters")
grid_search.fit(X, y)
best_rf = grid_search.best_estimator_

# Print Grid Search results
print("\nGrid Search Results:")
print(f"Best parameters: {grid_search.best_params_}")
print(f"Best score: {-grid_search.best_score_:.4f} MSE")

# Get CV results
cv_results = pd.DataFrame(grid_search.cv_results_)
cv_results = cv_results.sort_values('rank_test_score')

# Print top 5 parameter combinations
print("\nTop 5 parameter combinations:")
print(cv_results[['params', 'mean_test_score', 'std_test_score']].head())

# Save the model
joblib.dump(best_rf, 'RandomForest_model.pkl')
print("\nModel saved as 'RandomForest_model.pkl'")

```

Starting GridSearchCV with updated parameters
Fitting 3 folds for each of 720 candidates, totalling 2160 fits

Grid Search Results:

Best parameters: {'max_depth': None, 'max_features': 'sqrt', 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 200}

Best score: 0.2288 MSE

Top 5 parameter combinations:

	params	mean_test_score \
1	{'max_depth': None, 'max_features': 'sqrt', 'm...	-0.228841
24	{'max_depth': None, 'max_features': 'sqrt', 'm...	-0.229396
29	{'max_depth': None, 'max_features': 'sqrt', 'm...	-0.229396
2	{'max_depth': None, 'max_features': 'sqrt', 'm...	-0.229559
23	{'max_depth': None, 'max_features': 'sqrt', 'm...	-0.229824

	std_test_score
1	0.044948
24	0.045774
29	0.045774
2	0.045040
23	0.046000

Model saved as 'RandomForest_model.pkl'

7) Evaluate Model

```
In [7]: y_pred = best_rf.predict(X_test)
mape = mean_absolute_percentage_error(y_test, y_pred) * 100
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)
print(f"\nModel Performance:")
print(f"MAPE: {mape:.2f}%")
print(f"RMSE: {rmse:.4f}")
print(f"R2 Score: {r2:.4f}")
```

Model Performance:

MAPE: 78.01%

RMSE: 0.2152

R2 Score: 0.9009

8)Generate SHAP Values and Feature Importance

```
In [8]: # SHAP Values
explainer = shap.Explainer(best_rf)
shap_values = explainer(X_train)

plt.figure(figsize=(12, 8))
shap.summary_plot(shap_values, X_train, plot_type='bar', show=False)
plt.title('SHAP Feature Importance')
plt.tight_layout()
plt.savefig('shap_summary_bar.png', bbox_inches='tight', dpi=300)
plt.close()

# Random Forest Feature Importance
feature_importance = pd.DataFrame({
    'feature': selected_features,
    'importance': best_rf.feature_importances_
})
feature_importance = feature_importance.sort_values('importance', ascending=False)

plt.figure(figsize=(12, 8))
```

```
plt.bar(range(len(feature_importance)), feature_importance['importance'])
plt.xticks(range(len(feature_importance)), feature_importance['feature'], rotation=45, ha='right')
plt.title('Random Forest Feature Importance')
plt.tight_layout()
plt.savefig('rf_feature_importance.png', bbox_inches='tight', dpi=300)
plt.close()
```

9) Predict Future Returns

```
In [9]: # Make predictions using current data
future_predictions = best_rf.predict(X_current)

# Create results DataFrame
results_df = pd.DataFrame({
    'Company_Name': company_names,
    'Predicted_Future_Return': np.expm1(future_predictions) * 100 # Convert back to percentage
})

# Add prediction intervals using out-of-bag estimation
predictions = np.array([tree.predict(X_current) for tree in best_rf.estimators_])
prediction_std = np.std(predictions, axis=0)

# Calculate confidence intervals (95%)
results_df['Prediction_Std'] = prediction_std
results_df['Lower_Bound'] = np.expm1(future_predictions - 1.96 * prediction_std) * 100
results_df['Upper_Bound'] = np.expm1(future_predictions + 1.96 * prediction_std) * 100

# Add feature importance for each prediction
top_features = feature_importance['feature'].head(5).tolist()
for feat in top_features:
    results_df[f'Feature_{feat}'] = current_data[feat]

# Sort by predicted return
results_df = results_df.sort_values('Predicted_Future_Return', ascending=False)

# Save detailed results
results_df.to_csv('predicted_future_returns.csv', index=False)
print("\nFuture predictions saved to 'predicted_future_returns.csv'")

# Create summary statistics
summary_stats = pd.DataFrame({
    'Metric': ['Mean Predicted Return', 'Median Predicted Return',
              'Std Dev of Predictions', 'Min Predicted Return',
              'Max Predicted Return'],
    'Value': [
        results_df['Predicted_Future_Return'].mean(),
        results_df['Predicted_Future_Return'].median(),
        results_df['Predicted_Future_Return'].std(),
        results_df['Predicted_Future_Return'].min(),
        results_df['Predicted_Future_Return'].max()
    ]
})

# Save summary statistics
summary_stats.to_csv('prediction_summary_stats.csv', index=False)
print("Summary statistics saved to 'prediction_summary_stats.csv'")
```

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

```
c:\Users\namsi\AppData\Local\Programs\Python\Python311\Lib\site-packages\sklearn\base.py:486: UserWarning: X has feature names, but DecisionTreeRegressor was fitted without feature names
  warnings.warn(
c:\Users\namsi\AppData\Local\Programs\Python\Python311\Lib\site-packages\sklearn\base.py:486: UserWarning: X has feature names, but DecisionTreeRegressor was fitted without feature names
  warnings.warn(
c:\Users\namsi\AppData\Local\Programs\Python\Python311\Lib\site-packages\sklearn\base.py:486: UserWarning: X has feature names, but DecisionTreeRegressor was fitted without feature names
  warnings.warn(
c:\Users\namsi\AppData\Local\Programs\Python\Python311\Lib\site-packages\sklearn\base.py:486: UserWarning: X has feature names, but DecisionTreeRegressor was fitted without feature names
  warnings.warn(
c:\Users\namsi\AppData\Local\Programs\Python\Python311\Lib\site-packages\sklearn\base.py:486: UserWarning: X has feature names, but DecisionTreeRegressor was fitted without feature names
  warnings.warn(
Future predictions saved to 'predicted_future_returns.csv'
Summary statistics saved to 'prediction_summary_stats.csv'
```